

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа

«Хэш-таблицы»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 20

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

1. Создать динамический массив из записей (в соответствии с вариантом), содержащий не менее 100 элементов. Для заполнения элементов массива использовать ДСЧ.
2. Предусмотреть сохранение массива в файл и загрузку массива из файла.
3. Предусмотреть возможность добавления и удаления элементов из массива (файла).

АНАЛИЗ ЛИНЕЙНОГО ПРОБИРОВАНИЯ

1. Ключ — строка даты, переводим в число и считаем хэш.
2. Идём по таблице от индекса хэша, сдвигаясь вправо до пустой ячейки.
3. Если дошли до конца — продолжаем с начала до исходного хэша.
4. Вставка/поиск линейно зависят от числа коллизий; при переполнении вставка невозможна.

КОД LinearMap.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_LINEAR_MAP_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_LINEAR_MAP_H

#include "Human.h"

struct LinearMap {
    Human* arr = nullptr;
    int size = 0;

    void createTable(int s);
    void clearTable();

    void show();
    int findIndex(const std::string&);

    void add(const Human&);
    void pop(const std::string&);
};
```

```
private:
    int hashFunction(const std::string& s);
};

extern int collision_count;

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_LINEAR_MAP_H
```

КОД LinearMap.cpp

```
#include "LinearMap.h"

#include <cmath>
#include <iostream>

int collision_count = 0;

void LinearMap::createTable(int s) {
    size = s;
    arr = new Human[size];
    for (int i = 0; i < size; ++i)
        arr[i] = Human{"", "", ""};
}

void LinearMap::clearTable() {
    delete[] arr;
    arr = nullptr;
    size = 0;
}

int LinearMap::hashFunction(const std::string &s) {
    if (size == 0)
        return 0;
    Human temp;
    int k = temp.dateToNumber(s);
    double a = M_PI - std::floor(M_PI);
    double frac = k * a - std::floor(k * a);
    double c = size * frac;
    return c;
}

void LinearMap::show() {
    for (int i = 0; i < size; i++) {
        std::cout << i << ' ';
        if (arr[i].getName().empty())
```

```

        std::cout << "empty\n";
    else
        std::cout << arr[i] << '\n';
    }
}

int LinearMap::findIndex(const std::string &s) {
    int hash = hashFunction(s);
    int i = hash;

    while (i < size && arr[i].getBirthday() != s)
        i++;

    if (i >= size) {
        i = 0;
        while (i < hash && arr[i].getBirthday() != s)
            i++;
        if (i >= hash)
            return -1;
        return i;
    }
    return i;
}

void LinearMap::add(const Human &s) {
    int i = hashFunction(s.getBirthday());
    int hash = i;

    if (arr[i].getName().empty()) {
        arr[i] = s;
        return;
    }

    while (i < size) {
        if (arr[i].getName().empty()) {
            arr[i] = s;
            return;
        }
        i++;
        collision_count++;
    }
    if (i >= size) {
        i = 0;
        while (i < hash) {
            if (arr[i].getName().empty()) {

```

```

        arr[i] = s;
        return;
    }
    i++;
    collision_count++;
}
if (i >= hash) {
    std::cout << "Table is full, element can't be added.\n";
}
}
}

void LinearMap::pop(const std::string &s) {
    int i = findIndex(s);
    if (i != -1)
        arr[i] = Human("", "", "");
}

```

АНАЛИЗ МЕТОДА ЦЕПОЧЕК

1. Ключ — строка даты, переводим в число и считаем хэш.
2. Каждый индекс — список; вставка добавляет элемент в список корзины.
3. Поиск/удаление — линейный проход по списку корзины.
4. Коллизии считаются как длина списка перед вставкой.

КОД ChainMap.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_CHAIN_MAP_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_CHAIN_MAP_H
#define UNI_EXPORT
#include "Human.h"
#include "../manual_1/lab11/unidirectional/unidirectional.h"

struct ChainMap {
    UniList<Human>** arr = nullptr;
    int size = 0;

    void createTable(int s);
    void clearTable();

    void show();
    int findIndex(const std::string&) ;

    void add(const Human&);
    void pop(const std::string&);

private:
    int hashFunction(const std::string& s);
    int findIndexByKey(UniList<Human>*, const std::string&);
    void deleteByDate(UniList<Human>*, const std::string&);
    void printListHuman(UniList<Human>*);
};

extern int collision_count;

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_CHAIN_MAP_H
```

КОД ChainMap.cpp

```
#include "ChainMap.h"

#include <cmath>
#include <iostream>

void ChainMap::createTable(int s) {
    size = s;
    arr = new UniList<Human>*[size];
    for (int i = 0; i < size; ++i)
        arr[i] = new UniList<Human>();
}

void ChainMap::clearTable() {
    if (arr == nullptr)
        return;
    for (int i = 0; i < size; ++i) {
        if (arr[i] != nullptr) {
            arr[i]->clear();
            delete arr[i];
            arr[i] = nullptr;
        }
    }
    delete[] arr;
    arr = nullptr;
    size = 0;
}

int ChainMap::hashFunction(const std::string &s) {
    if (size == 0)
        return 0;
    Human temp;
    int k = temp.dateToNumber(s);
    double pi = std::acos(-1.0);
    double a = pi - std::floor(pi);
    double frac = k * a - std::floor(k * a);
    double c = size * frac;
    return c;
}

int ChainMap::findIndexByKey(UniList<Human>* list, const std::string &key) {
    if (list == nullptr)
        return -1;
    auto node = list->head;
```

```

    for (int i = 0; node != nullptr; i++) {
        if (node->data.getBirthDay() == key)
            return i;
        node = node->next;
    }
    return -1;
}

void ChainMap::deleteByDate(UniList<Human>* list, const std::string &key) {
    if (list == nullptr || list->head == nullptr) {
        std::cout << "List is empty\n";
        return;
    }
    auto node = list->head;
    int ind = 0;
    while (node != nullptr) {
        if (node->data.getBirthDay() == key)
            break;
        node = node->next;
        ++ind;
    }
    if (node == nullptr) {
        std::cout << "Element with date \"" << key << "\" not found\n";
        return;
    }
    list->remove(ind);
}

void ChainMap::printListHuman(UniList<Human>* list) {
    if (list == nullptr)
        return;
    auto node = list->head;
    while (node != nullptr) {
        std::cout << node->data << '\n';
        node = node->next;
    }
}

void ChainMap::show() {
    for (int i = 0; i < size; i++) {
        std::cout << "Bucket index: " << i << '\n';
        printListHuman(arr[i]);
    }
}

```



```
int ChainMap::findIndex(const std::string &to_find) {  
    int hash = hashFunction(to_find);  
    return hash;  
}
```

```
void ChainMap::add(const Human &to_add) {  
    int ind = hashFunction(to_add.getBirthDay());  
    if (arr[ind] == nullptr)  
        return;  
    collision_count += arr[ind]->size;  
    Human tmp = to_add;  
    arr[ind]->pushBack(tmp);  
}
```

```
void ChainMap::pop(const std::string &to_delete) {  
    int ind = findIndex(to_delete);  
    if (ind < 0 || ind >= size)  
        return;  
    int index_list = findIndexByKey(arr[ind], to_delete);  
    if (index_list != -1)  
        deleteByDate(arr[ind], to_delete);  
}
```

КОД main.cpp

```
#include "Human.h"
#include "LinearMap.h"
#include "ChainMap.h"
#include <iostream>

int main() {
    Human humans[10];
    for (auto & human : humans) human = Human();

    LinearMap linear;
    linear.createTable(10);

    for (auto & human : humans) linear.add(human);

    std::cout << "Find index for " << humans[1].getBirthday() << ": "
        << linear.findIndex(humans[1].getBirthday()) << '\n';
    linear.show();

    linear.pop(humans[1].getBirthday());
    std::cout << "After delete:\n";
    linear.show();
    std::cout << "Linear collisions: " << collision_count << "\n\n";
    linear.clearTable();

    ChainMap chain;
    chain.createTable(5);

    collision_count = 0;
    for (auto & human : humans) chain.add(human);

    std::cout << "Find index for " << humans[0].getBirthday() << ": "
        << chain.findIndex(humans[0].getBirthday()) << '\n';
    chain.show();

    chain.pop(humans[0].getBirthday());
    std::cout << "After delete:\n";
    chain.show();
    std::cout << "Chain collisions: " << collision_count << '\n';

    chain.clearTable();
    return 0;
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Find index for 14.2.1983: 3

- 0 Bid Komupu | 23.7.1977 | HT485535731
- 1 Reli Dutes | 5.5.2006 | HV811699284
- 2 Zuv Lakifad | 19.11.1974 | QC162683460
- 3 Vegel Vahiya | 14.2.1983 | RS101921262
- 4 Yoq Jafiku | 4.10.2009 | OB370246166
- 5 Fise Zemex | 8.6.1976 | LH987381618
- 6 Haho Siwo | 6.10.2009 | JP404816650
- 7 Say Gabigi | 1.12.2026 | MX825742444
- 8 Wuko Jexix | 17.8.1983 | YU618280114
- 9 Qebopi Coloyemuj | 25.2.2002 | ZO628647068

After delete:

- 0 Bid Komupu | 23.7.1977 | HT485535731
- 1 Reli Dutes | 5.5.2006 | HV811699284
- 2 Zuv Lakifad | 19.11.1974 | QC162683460
- 3 empty
- 4 Yoq Jafiku | 4.10.2009 | OB370246166
- 5 Fise Zemex | 8.6.1976 | LH987381618
- 6 Haho Siwo | 6.10.2009 | JP404816650
- 7 Say Gabigi | 1.12.2026 | MX825742444
- 8 Wuko Jexix | 17.8.1983 | YU618280114
- 9 Qebopi Coloyemuj | 25.2.2002 | ZO628647068

Linear collisions: 12

Find index for 25.2.2002: 4

Bucket index: 0

Zuv Lakifad | 19.11.1974 | QC162683460

Bucket index: 1

Veqel Vahiya | 14.2.1983 | RS101921262

Yoq Jafiku | 4.10.2009 | OB370246166

Bucket index: 2

Fise Zemex | 8.6.1976 | LH987381618

Bid Komupu | 23.7.1977 | HT485535731

Bucket index: 3

Haho Siwo | 6.10.2009 | JP404816650

Say Gabigi | 1.12.2026 | MX825742444

Wuko Jexix | 17.8.1983 | YU618280114

Bucket index: 4

Qebopi Coloyemuj | 25.2.2002 | ZO628647068

Reli Dutes | 5.5.2006 | HV811699284

After delete:

Bucket index: 0

Zuv Lakifad | 19.11.1974 | QC162683460

Bucket index: 1

Veqel Vahiya | 14.2.1983 | RS101921262

Yoq Jafiku | 4.10.2009 | OB370246166

Bucket index: 2

Fise Zemex | 8.6.1976 | LH987381618

Bid Komupu | 23.7.1977 | HT485535731

Bucket index: 3

Haho Siwo | 6.10.2009 | JP404816650

Say Gabigi | 1.12.2026 | MX825742444

Wuko Jexix | 17.8.1983 | YU618280114

Bucket index: 4

Reli Dutes | 5.5.2006 | HV811699284

Chain collisions: 6